POLYTECH
NICE SOPHIA

# Equipe O

# Conception logicielle

# SI4 2025/2026




**BOULET Olivier - PO**

**KARBAL Adam - OPS**

**MADANDAR ARANI Ali – PO & QA**

**MENOU Clement - QA**

**GUERRAOUI Ismail - SA**

# Sommaire :

Glossaire ..... 4

- Définitions des concepts clés du domaine ..... 4

Diagramme de cas d'utilisation ..... 6

- Présentation des acteurs ..... 6
- Principaux cas d'utilisation ..... 6

Diagramme de classes ..... 8

- Justification des choix ..... 8

Diagramme de séquence ..... 9

- Cas d'utilisation « passer une commande » avec paiement externe ..... 9

Maquette ..... 10

- Esquisse de l'interface liée au scénario de commande ..... 10

# Glossaire

- Définitions des concepts clés du domaine

## I. Acteurs

| Terme | Définition |
|---|---|
| **User (Utilisateur)** | Personne non enregistrée. Peut parcourir restaurants et plats, filtrer, mais ne commande pas. |
| **Campus Member (membre du campus)** | Sous-type de *User*, authentifié. Peut créer une commande, ajouter des plats, valider et payer, suivre l'état. |
| **Restaurant Owner (Gérant)** | Gère son restaurant : création/édition de plats, paramétrage des horaires et capacités. |

## II. Concepts clés

| Terme | Définition |
|---|---|
| **Account (Compte)** | Profil d'un *Campus Member* (id, éventuel crédit étudiant). |
| **Restaurant** | Établissement avec nom, type de cuisine, type d'établissement, planning et capacité par créneau de 30min. |
| **RestaurantSchedule (Planning)** | Créneaux d'ouverture |
| **Dish (Plat)** | Nom, prix, catégorie (entrée/plat/dessert/boisson), type (pizza, salade…), tags (sans-gluten...), toppings éventuels. |
| **Topping (Supplément)** | Option ajoutable au plat (ex. fromage, frites, sauce). |
| **Order (Commande)** | Ensemble de plats d'un seul restaurant, non vide. Porte l'état (créée, validée, payée…). Appartient à un *Account*. |
| **RestaurantRepository** | Collection de restaurants. |
| **RestaurantFilter** | Applique des critères (tags, type cuisine/étab., prix, disponibilité). |
| **RestauranService** | Expose les restaurants disponibles. |
| **OrderManager** | Crée/ajoute/supprime/valide/suit/ annule la commande. |
| **RestaurantManager** | Gère plats, planning, capacités. |

## III. Paiement & règles

| Terme | Définition |
|---|---|
| **Payment (Paiement)** | Enregistrement du règlement d'une commande : montant, méthode, statut. Rattaché à une Order |
| **PaymentService** | Service (externe) qui exécute le paiement et renvoie succès/échec. |
| **PaymentMethod** | Méthode utilisée : externe ou crédit étudiant. |
| **Credit/Paiement interne** | Montant alloué (p.ex. 20 €) utilisable comme moyen de paiement sur le compte utilisateur, un solde |
| **Capacity (Capacité)** | Nombre max de commandes par demi-heure accepté par un restaurant |

# Diagramme de cas d’utilisation

- ## Présentation des acteurs

**User** : utilisateur non authentifié, peut naviguer dans les restaurants et plats disponibles.

**Campus Member** : sous-type de *User*, il est authentifié et peut passer une commande complète (ajout d’items, choix du lieu, paiement, suivi).

**Restaurant Owner** : responsable d’un restaurant, il peut enregistrer son restaurant, ajouter/éditer ses plats et gérer ses disponibilités.

- ## Principaux cas d’utilisation

Pour le **User** :

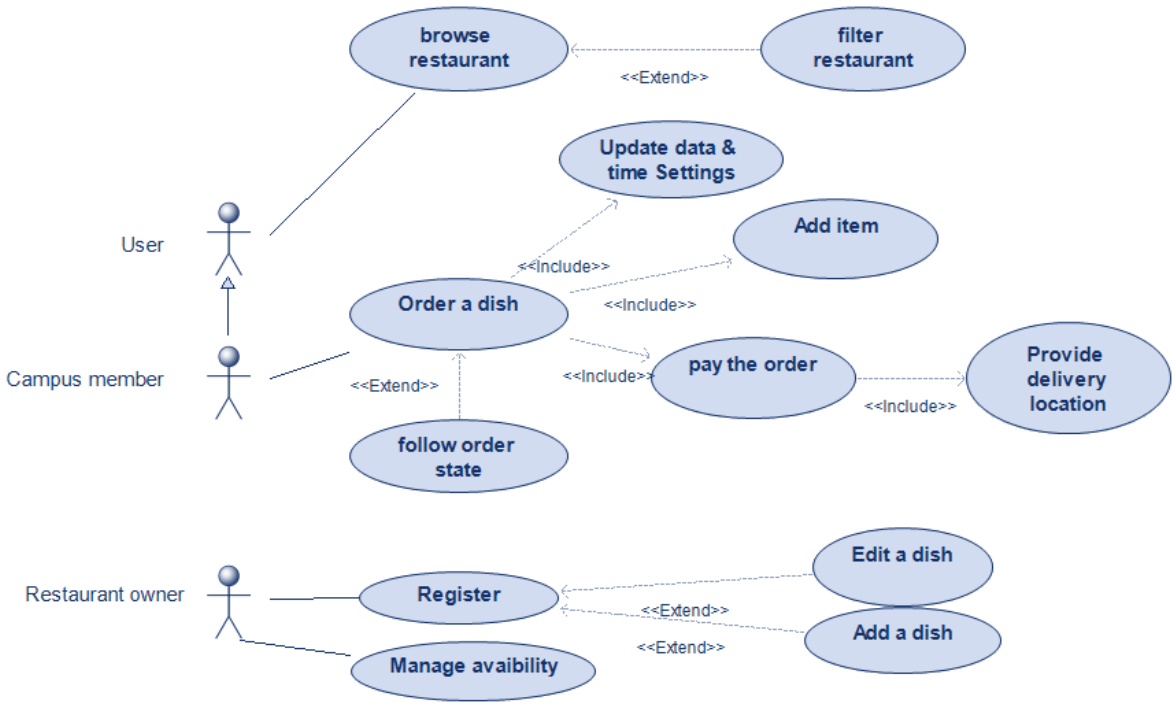
- **Browse restaurant** : consulter la liste des restaurants disponibles.
- **Filter restaurant** : appliquer des filtres (prix, type de cuisine, horaires, options alimentaires). (*Extend* de *Browse restaurant*).

Pour l’acteur **Campus Member :**

- **Order a dish** : créer une commande en sélectionnant un restaurant et des plats.
- *Include* :
    - **Add item** : ajouter un plat à la commande.
    - **Update data & time settings** : définir la date et le créneau de livraison possible.
    - **Provide delivery location** : choisir un lieu de livraison prédéfini.
    - **Pay the order** : effectuer le paiement (externe ou crédit étudiant).
- **Follow order state** : consulter l’avancement de sa commande (*Extend* de *Order a dish*).

Pour l’acteur **Restaurant Owner :**

- **Register** : inscrire un restaurant dans le système.
- *Extend* : **Add a dish** (ajout de nouveaux plats).
- *Extend* : **Edit a dish** (modifier les informations d’un plat).
- **Manage availability** : définir ou modifier les créneaux horaires et la capacité (nb max de commandes par demi-heure).


browse restaurant
filter restaurant
<<Extend>>
Update data & time Settings
User
Add item
<<Include>>
Order a dish
<<Include>>
Campus member
<<Include>>
pay the order
Provide delivery location
<<Include>>
<<Extend>>
follow order state
Edit a dish
Restaurant owner
Register
<<Extend>>
Add a dish
<<Extend>>
Manage avaibility

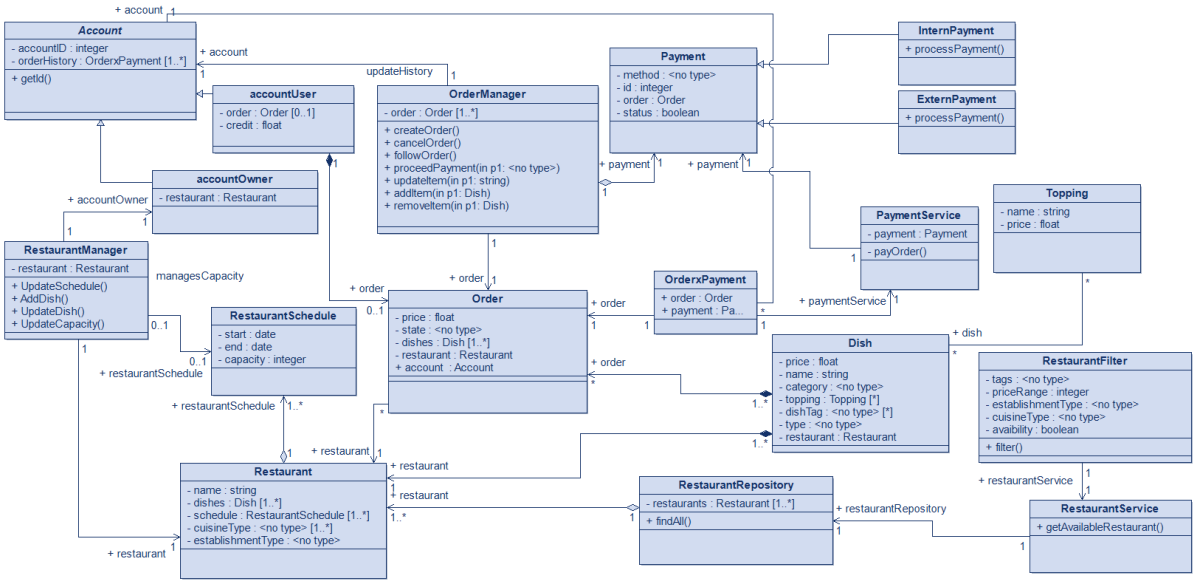
# Diagramme de classes

- ## Justification des choix

A une échelle macroscopique, on a décidé de segmenter l'architecture générale en micro-services, un qui se charge des restaurants, un qui s'occupe de la commande et un qui s'occupe du paiement. A l'intérieur de ces micro-services, nous avons utilisé le patron de conception MVC.

Les classes du modèle représentent les entités métier : *Restaurant*, *Dish*, *Order*, *Account*, *Payment*, *RestaurantSchedule*, etc., qui encapsulent les données essentielles et leurs relations (par exemple, un restaurant possède plusieurs plats et plusieurs créneaux horaires, un plat peut avoir des toppings, catégories, types et tags). Le contrôleur est incarné par des classes de gestion comme *OrderManager* et *RestaurantManager*, chargées de la logique métier (créer/annuler une commande, mettre à jour les plats ou la capacité d'un restaurant). Enfin, la vue est implicitement représentée via les services (*RestaurantService*, *PaymentService*, *RestaurantFilter*), qui exposent des méthodes permettant aux utilisateurs ou à l'interface de consulter et manipuler les données.

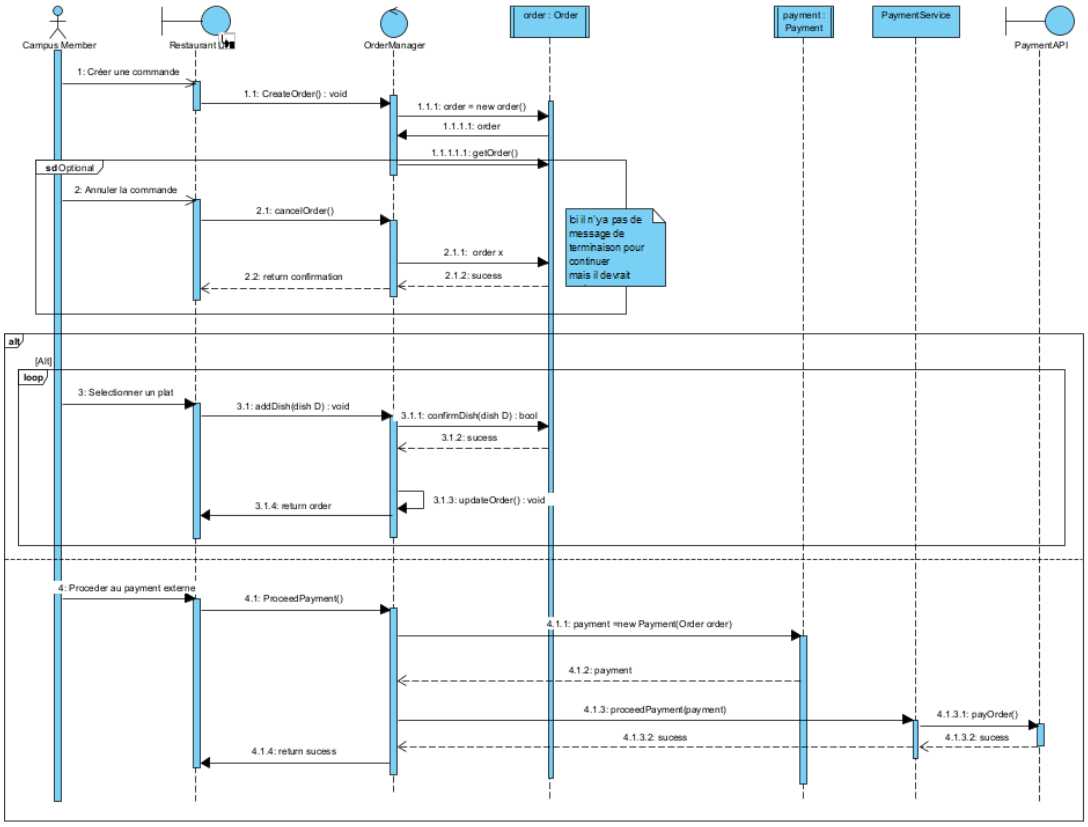
Le choix de séparer clairement les classes métier (ex. *Order*, *Dish*) des classes qui vont les gérer (*OrderManager*, *RestaurantManager*) et des services (*PaymentService*, *RestaurantService*) permet une meilleure extensibilité : le modèle reste indépendant de la logique de traitement et des interfaces.



# Diagramme de séquence

- ## Cas d’utilisation « passer une commande » avec paiement externe

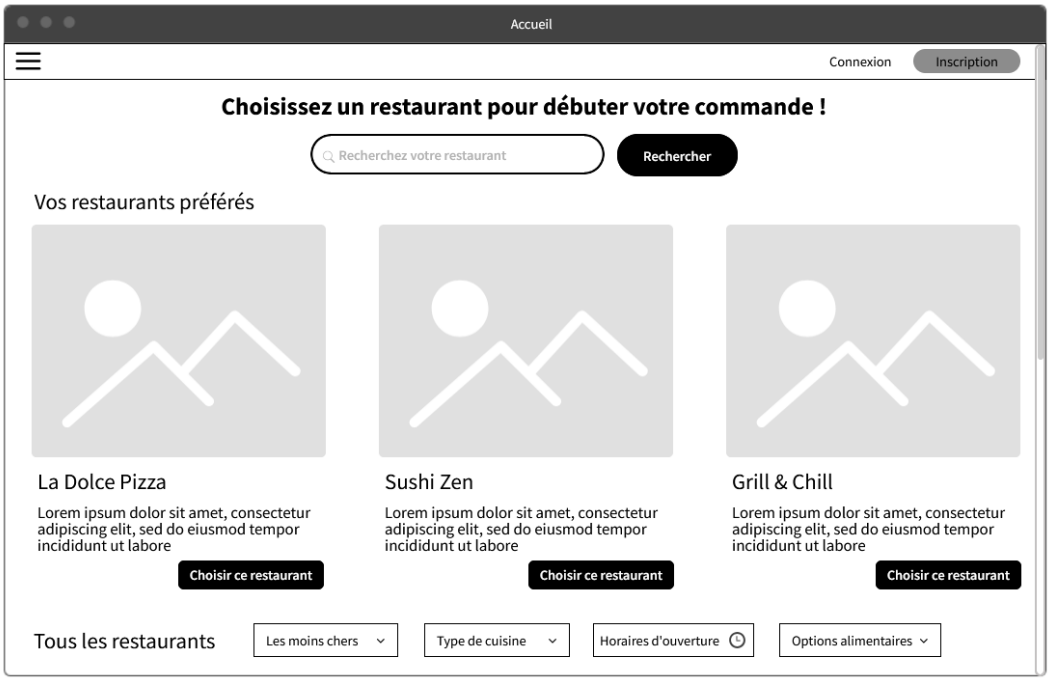
  - Exigences concernées : C5,C6,C7,P1,P2
  - Cas d’erreur présents : Dans l’alt, on ne peut pas procéder au paiement s’il n’y a rien dans la commande



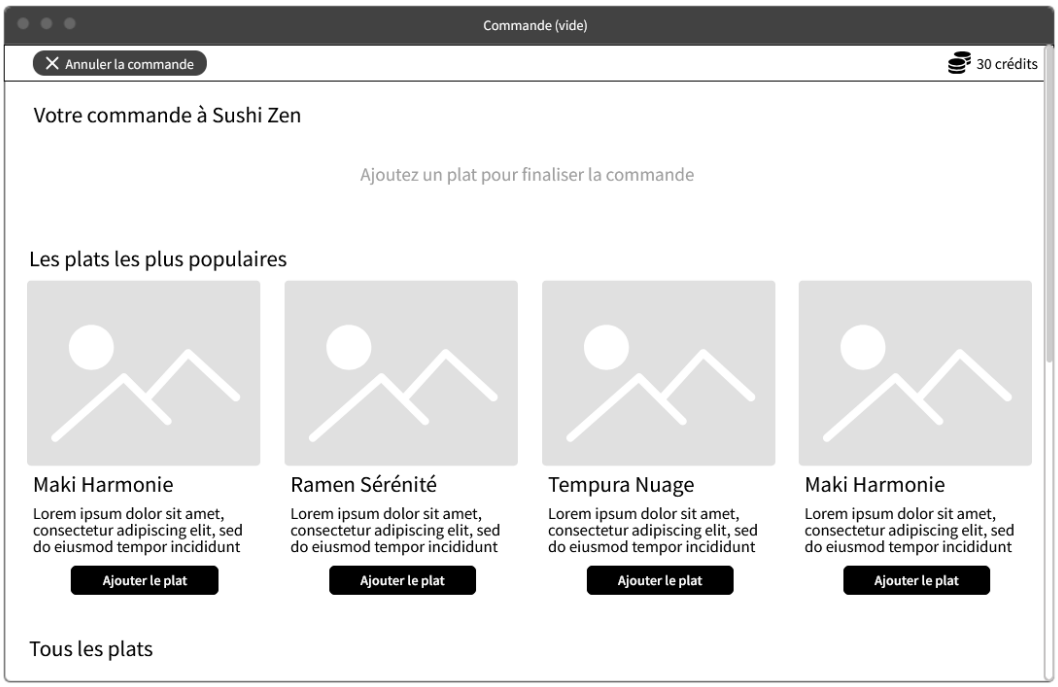
# Maquette

- ## Esquisse de l’interface liée au scénario de commande

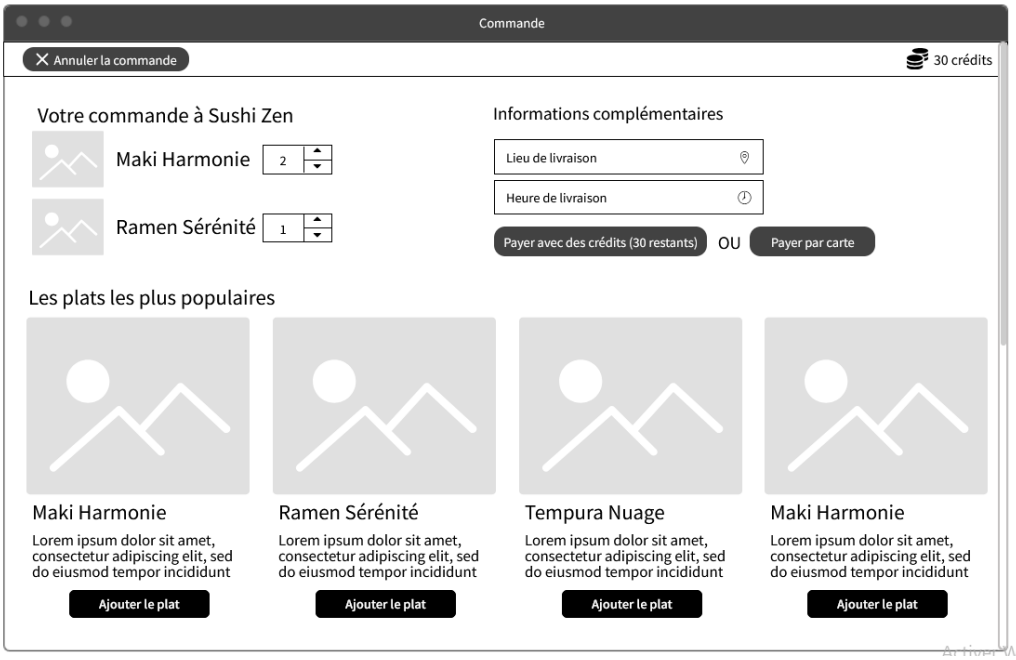
Accueil



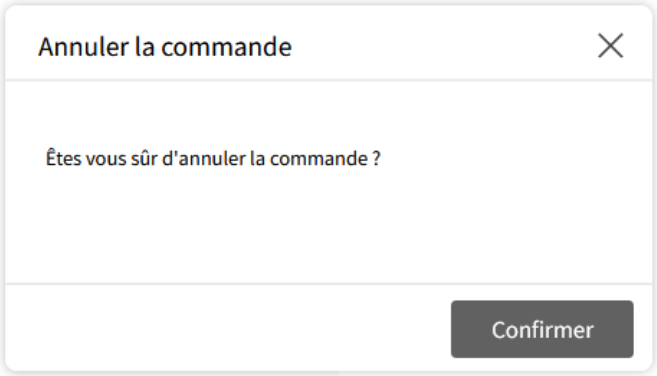

Interface de commande (vide)




Interface de commande




Popup d’annulation de commande




Popup d’erreur en cas de créneaux horaire indisponible

